

INF672 · MACHINE LEARNING · PROBABILIDAD

Dos formas de medir el azar

El enfoque clásico (contar) y el frecuentista (repetir) — con simulaciones en Python

Campo	Detalle
Asignatura	INF672 · Machine Learning
Guía	GL6 — Probabilidad: enfoque clásico y frecuentista
Unidad	Fundamentos matemáticos del aprendizaje (probabilidad)
Modalidad	Laboratorio práctico en Python (Jupyter/NumPy/Matplotlib)
Docente	M.Sc. Huáscar Fedor Gonzales Guzmán
Carrera	Ingeniería Informática · Facultad de Ciencias Puras · UATF

Esta guía reconstruye, paso a paso, el cuaderno de la práctica. Cada bloque de código va acompañado de dos cosas que un notebook no siempre trae: una explicación conceptual ampliada de la idea que se está probando, y un desglose detallado de **qué hace el código, línea por línea**. Ejecuta las celdas en orden (Shift + Enter) y contrasta tus salidas con las que aquí se muestran.

01 Objetivos de aprendizaje

Al finalizar esta práctica, el estudiante será capaz de:

- Distinguir las dos interpretaciones fundamentales de la probabilidad: la **clásica** (a priori, por conteo) y la **frecuentista** (a posteriori, por repetición).
- Aplicar la **regla de Laplace** enumerando correctamente el **espacio muestral equiprobable** y contando casos favorables.
- Reconocer la **condición de simetría** que sostiene al enfoque clásico y explicar por qué su ausencia lo invalida.
- Implementar **simulaciones de Monte Carlo** con NumPy para estimar probabilidades a partir de datos.
- Visualizar la **convergencia de la frecuencia relativa** (ley de los grandes números) y la varianza de la estimación.
- Conectar ambos enfoques con ideas nucleares del machine learning: prior uniforme, estimación por máxima verosimilitud, el valor de los datos y el anticipo del sobreajuste.

02 Entorno y preparación

El laboratorio usa cuatro herramientas del ecosistema científico de Python: **NumPy** (arreglos y azar vectorizado), **Matplotlib** (gráficos), y dos utilidades de la librería estándar: `itertools.product` (para generar combinaciones) y `collections.Counter` (para contar frecuencias). La celda siguiente carga todo y fija la identidad visual de la materia. Ejecútala una sola vez.

In [1]:

```
import numpy as np
import matplotlib.pyplot as plt
import matplotlib as mpl
from itertools import product
from collections import Counter

# --- Identidad visual INF672 "cuaderno científico" ---
INDIGO = "#4F46E5" # primario
PURPURA = "#7E22CE" # apoyo
CORAL = "#F43F5E" # errores / advertencias / énfasis
TINTA = "#111827" # texto
CELDA = "#F9FAFB" # fondo de celda
HAIRLINE = "#E5E7EB"

mpl.rcParams.update({
    "font.family": ["DejaVu Sans", "sans-serif"],
    "font.size": 11,
    "axes.facecolor": CELDA,
    "figure.facecolor": "white",
    "axes.grid": True,
    "grid.color": HAIRLINE,
})

# Semilla: hace el azar reproducible (mismo resultado en cada corrida).
rng = np.random.default_rng(672)
print("Entorno listo. Paleta INF672 cargada.")
```

Out:

```
Entorno listo. Paleta INF672 cargada.
```

CÓMO FUNCIONA EL CÓDIGO

- Las cuatro primeras líneas *importan* las librerías y les dan alias cortos (`np`, `plt`). Escribir `np.array(...)` en vez de `numpy.array(...)` es la convención universal de la comunidad.
- El bloque de constantes de color guarda la paleta como cadenas hexadecimales; centralizarlas evita repetir el mismo código de color en cada gráfico.
- `mpl.rcParams.update(...)` modifica los *ajustes por defecto* de Matplotlib para toda la sesión: fija la tipografía, el color de fondo de los ejes y una rejilla tenue. Así, cada figura posterior hereda el mismo estilo sin repetir instrucciones.
- `np.random.default_rng(672)` crea un **generador de números aleatorios** con una **semilla** fija (672). La semilla es la clave: garantiza que la secuencia “aleatoria” sea idéntica cada vez que ejecutes el cuaderno, de modo que tus resultados coincidan con los de esta guía. Es la forma moderna y recomendada de generar azar en NumPy, preferible al antiguo `np.random.seed`.

03 Marco conceptual: ¿qué significa “probabilidad”?

Probabilidad es una palabra que usamos a diario con soltura y que, sin embargo, esconde una pregunta sin respuesta única: cuando afirmamos que algo tiene un 70 % de probabilidad, *¿qué estamos diciendo exactamente sobre el mundo?* A lo largo de la historia se consolidaron varias respuestas. Esta guía trabaja las dos más antiguas y operativas, porque entender la diferencia entre ellas es entender dónde nace el machine learning.

INTERPRETACIÓN CLÁSICA (A PRIORI)

Formalizada por Laplace en el siglo XVIII. La probabilidad es una propiedad de la **estructura** del experimento: si por **simetría** hay N resultados igualmente posibles, cada uno vale $1/N$. Se calcula *antes* de experimentar, con pura aritmética. Es exacta... cuando la simetría existe.

INTERPRETACIÓN FRECUENTISTA (A POSTERIORI)

Consolidada en los siglos XIX–XX. La probabilidad es una propiedad de la **repetición**: la fracción de veces que ocurre un evento cuando el experimento se repite un número enorme de veces. Se mide *durante* el experimento y siempre entrega una **estimación**, nunca el valor perfecto.

Existe una tercera vía, la **interpretación bayesiana o subjetiva** (la probabilidad como grado de creencia que se actualiza con la evidencia), que retomaremos más adelante en el curso. Por ahora nos quedamos con el contraste esencial:

	Enfoque clásico	Enfoque frecuentista
El gesto	Contar	Repetir y observar
¿Cuándo?	Antes de experimentar	Durante el experimento
Necesita	Simetría (equiprobabilidad)	Muchísimas repeticiones
Entrega	El valor exacto	Una estimación
Se rompe si	No hay simetría	Hay pocos datos
Experimento	Suma de dos dados	Moneda y chincheta

La idea central de todo el cuaderno: el enfoque clásico es exacto pero **frágil** (se rompe cuando no hay simetría), y justo ahí toma el relevo el frecuentista — que es también donde vive el aprendizaje automático.

04 Parte 1 · El enfoque clásico: contar sin tirar

El enfoque clásico calcula la probabilidad **antes de experimentar**, con aritmética. Su fórmula —la **regla de Laplace**— es célebre por su sencillez:

$$P(A) = \frac{\text{número de resultados favorables}}{\text{número de resultados posibles}}$$

“Favorables” no significa “buenos”, sino “los que cumplen la condición que preguntas”. Contar arriba, contar abajo, dividir. Eso es todo — **pero con una condición que lo es todo**: los resultados que cuentas abajo deben ser igualmente posibles.

La pregunta y la trampa

¿Cuál es la probabilidad de que la suma de dos dados sea 7?

Antes de calcular, conviene apostar mentalmente. Un razonamiento muy común —y equivocado— es este:

RESPUESTA INGENUA

“Las sumas posibles van de 2 a 12, o sea 11 valores distintos. Entonces $P(7) = 1/11 \approx 0.091$.” Esta respuesta está **mal**, y lo bonito es que el propio enfoque clásico, bien aplicado, revela el error.

Paso 1 — Enumerar el espacio muestral

El error del “1/11” es suponer que las **sumas** son igual de posibles. No lo son. Lo que sí es equiprobable es cada **par de dados** (dado 1, dado 2). Hay $6 \times 6 = 36$ pares, y cada uno tiene exactamente la misma probabilidad, 1/36. Generemos esos 36 pares.

In [2]:

```
# Todos los pares posibles (d1, d2) con cada dado de 1 a 6.
espacio_muestral = list(product(range(1, 7), repeat=2)) # 36 pares
print(f"Total de resultados equiprobables: {len(espacio_muestral)}\n")

# Los mostramos como una rejilla 6x6 para verlos todos de un vistazo.
for d1 in range(1, 7):
    fila = " ".join(f"({d1},{d2})" for d2 in range(1, 7))
    print(fila)
```

Out:

```
Total de resultados equiprobables: 36

(1,1) (1,2) (1,3) (1,4) (1,5) (1,6)
(2,1) (2,2) (2,3) (2,4) (2,5) (2,6)
(3,1) (3,2) (3,3) (3,4) (3,5) (3,6)
(4,1) (4,2) (4,3) (4,4) (4,5) (4,6)
(5,1) (5,2) (5,3) (5,4) (5,5) (5,6)
(6,1) (6,2) (6,3) (6,4) (6,5) (6,6)
```

CÓMO FUNCIONA EL CÓDIGO

- `product(range(1, 7), repeat=2)` calcula el **producto cartesiano** del conjunto $\{1,2,3,4,5,6\}$ consigo mismo: todas las parejas ordenadas posibles. `range(1, 7)` produce 1,2,3,4,5,6 (el 7 queda excluido, porque en Python el límite superior es abierto), y `repeat=2` indica "dos dados". Envolverlo en `list(...)` materializa las 36 tuplas en una lista.
- `len(espacio_muestral)` confirma que son 36. La **f-string** (la cadena que empieza con `f"..."`) permite incrustar variables entre llaves; `\n` inserta un salto de línea.
- El `for` externo recorre las seis filas (dato 1). Dentro, la **comprensión de generador** `f"({d1},{d2})"` `for d2 in range(1, 7)` arma las seis casillas de esa fila, y `.join(...)` las une con dos espacios. El resultado es la rejilla 6×6.

IDEA CLAVE

Cada una de esas 36 casillas es un resultado con probabilidad $1/36$. **Ahí** vive la equiprobabilidad, no en las sumas. Identificar bien el espacio muestral equiprobable es el enfoque clásico.

Paso 2 — Contar los casos favorables

Ahora aplicamos la fórmula. Para cada suma posible (de 2 a 12) contamos cuántos pares la producen y dividimos entre 36. La suma 7 se logra de seis maneras —(1,6),(2,5),(3,4),(4,3),(5,2),(6,1)—, mientras que la suma 2 solo de una: (1,1).

In [3]:

```
# Contamos cuántos pares dan cada suma.
conteo = Counter(d1 + d2 for d1, d2 in espacio_muestral)

print(f"{'Suma':>5} {'Pares favorables':>17} {'Probabilidad':>14}")
print("-" * 40)
for suma in range(2, 13):
    favorables = conteo[suma]
    prob = favorables / 36
    print(f"{'suma':>5} {'favorables':>17} {'prob:>10.4f} ({favorables}/36)")

print(f"\nP(suma = 7) = {conteo[7]}/36 = {conteo[7]/36:.4f}")
```

Out:

Suma	Pares favorables	Probabilidad
2	1	0.0278 (1/36)
3	2	0.0556 (2/36)
4	3	0.0833 (3/36)
5	4	0.1111 (4/36)
6	5	0.1389 (5/36)
7	6	0.1667 (6/36)
8	5	0.1389 (5/36)
9	4	0.1111 (4/36)
10	3	0.0833 (3/36)
11	2	0.0556 (2/36)
12	1	0.0278 (1/36)

P(suma = 7) = 6/36 = 0.1667

CÓMO FUNCIONA EL CÓDIGO

- Counter(d1 + d2 for d1, d2 in espacio_muestral) recorre las 36 tuplas, **desempaqueta** cada una en d1 y d2, calcula la suma y las cuenta. Counter es un diccionario especializado: la clave es la suma y el valor es cuántas veces apareció. Así, conteo[7] devuelve 6 directamente.
- Los especificadores de formato como >5 (alinear a la derecha en 5 espacios) y .4f (cuatro decimales) sirven para que la tabla salga en columnas prolijas. Combinar >10.4f aplica ambas cosas a la vez.
- La probabilidad se obtiene con favorables / 36, que es exactamente la regla de Laplace aplicada a este espacio muestral.

La probabilidad real de sacar 7 es $6/36 \approx 16.7\%$, no el 9.1% que sugería el “1/11”. El 7 es, de hecho, **la suma más probable** —no por casualidad, sino porque es la que más pares distintos puede formar.

Paso 3 — La forma de la distribución

Graficadas las 11 probabilidades, aparece una silueta triangular con pico en el 7: la marca de que las sumas centrales tienen muchas más formas de ocurrir que los extremos.

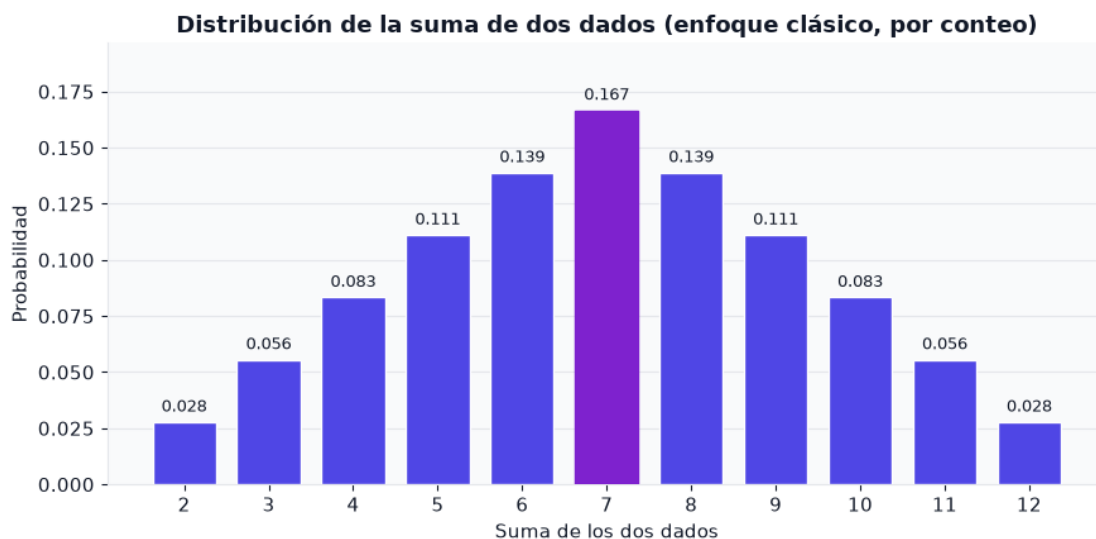
In [4]:

```
sumas = list(range(2, 13))
probs = [conteo[s] / 36 for s in sumas]

fig, ax = plt.subplots(figsize=(9, 4.5))
barras = ax.bar(sumas, probs, color=INDIGO, edgecolor="white", width=0.75)
barras[sumas.index(7)].set_color(PURPURA) # resalta el máximo

for s, p in zip(sumas, probs):
    ax.text(s, p + 0.003, f"{p:.3f}", ha="center", va="bottom", fontsize=9)

ax.set_title("Distribución de la suma de dos dados (por conteo)")
ax.set_xlabel("Suma de los dos dados"); ax.set_ylabel("Probabilidad")
ax.set_xticks(sumas)
plt.tight_layout(); plt.show()
```



Out [4] · Distribución triangular de la suma de dos dados; en púrpura, el máximo en 7.

CÓMO FUNCIONA EL CÓDIGO

- Se construyen dos listas paralelas: *sumas* (2 a 12) y *probs*, esta última mediante una **comprensión de lista** que divide cada conteo entre 36.
- `plt.subplots()` crea la figura (*fig*) y el sistema de ejes (*ax*); casi todo se dibuja sobre *ax*. `ax.bar(...)` dibuja las barras y devuelve la lista de objetos-barra.
- `sumas.index(7)` localiza la posición del 7 en la lista (índice 5) para pintar esa barra de púrpura. El `for` con `ax.text(...)` escribe el valor numérico encima de cada barra; `zip(sumas, probs)` recorre ambas listas en paralelo.
- `tight_layout()` ajusta márgenes para que nada se recorte, y `show()` renderiza la figura.

Paso 4 — Ver por qué, con un mapa de calor

El gráfico de barras muestra *qué* pasa; el mapa de calor muestra *por qué*. Cada celda es un par (d_1, d_2) coloreada según su suma. Las **diagonales** agrupan pares con la misma suma —y la diagonal del 7 es la más larga (6 celdas), mientras que el 2 y el 12 quedan atrapados en una sola esquina.

In [5]:

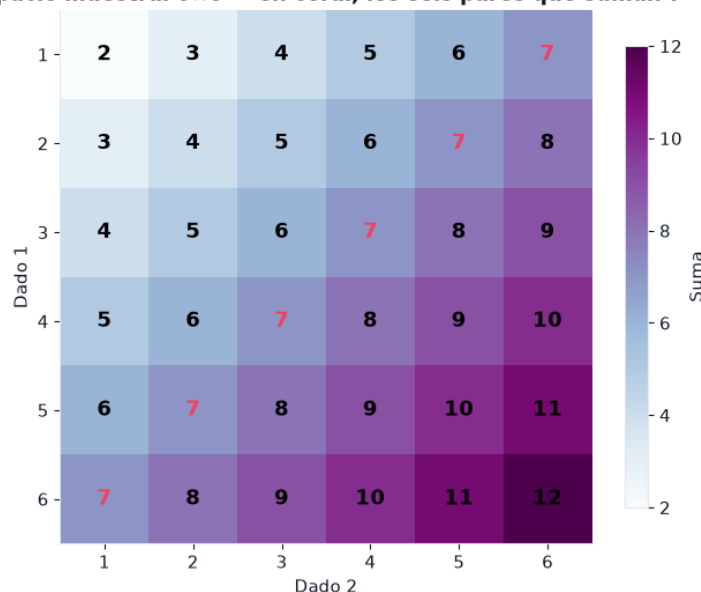
```
# Matriz 6x6 donde cada celda es la suma d1 + d2.
matriz = np.array([[d1 + d2 for d2 in range(1, 7)] for d1 in range(1, 7)])

fig, ax = plt.subplots(figsize=(6.5, 5.5))
im = ax.imshow(matriz, cmap="BuPu", origin="upper")

for i in range(6):
    for j in range(6):
        es_siete = (matriz[i, j] == 7)
        ax.text(j, i, matriz[i, j], ha="center", va="center",
                color=CORAL if es_siete else "black", fontweight="bold")

ax.set_xticklabels(range(1, 7)); ax.set_yticklabels(range(1, 7))
ax.set_xlabel("Dado 2"); ax.set_ylabel("Dado 1")
plt.colorbar(im, ax=ax, label="Suma"); plt.show()
```

Espacio muestral 6×6 — en coral, los seis pares que suman 7



Out [5] · Espacio muestral 6×6; en coral, los seis pares que suman 7 (la diagonal más larga).

CÓMO FUNCIONA EL CÓDIGO

- La **comprensión de lista anidada** construye una tabla 6×6 de sumas y `np.array(...)` la convierte en una matriz de NumPy, que se indexa con `matriz[i, j]`.
- `ax.imshow(matriz, cmap="BuPu")` pinta la matriz como imagen: cada valor se mapea a un color de la paleta azul-púrpura, y los valores mayores (sumas altas) salen más oscuros.
- El doble `for` recorre filas (`i`) y columnas (`j`) y escribe el número dentro de cada celda. La **expresión condicional** CORAL `if es_siete else "black"` pinta de coral solo los que suman 7.
- `plt.colorbar(...)` añade la barra de referencia de color a la derecha.

LO QUE REVELA LA FIGURA

Se ve de un golpe: el 7 tiene la diagonal más larga. Contar bien el espacio muestral equiprobable es el enfoque clásico; equivocarse al identificarlo (contar sumas en vez de pares) es lo que producía el "1/11".

Paso 5 — Verificación: ¿y si tiramos de verdad?

Todo lo anterior fue aritmética *a priori*: no lanzamos ni un dado. Ahora, como puente hacia la Parte 2, **simulamos** 50 000 tiradas reales y superponemos las frecuencias observadas sobre las probabilidades calculadas por conteo. El conteo ya había *predicho* el resultado; la simulación solo lo *confirma*.

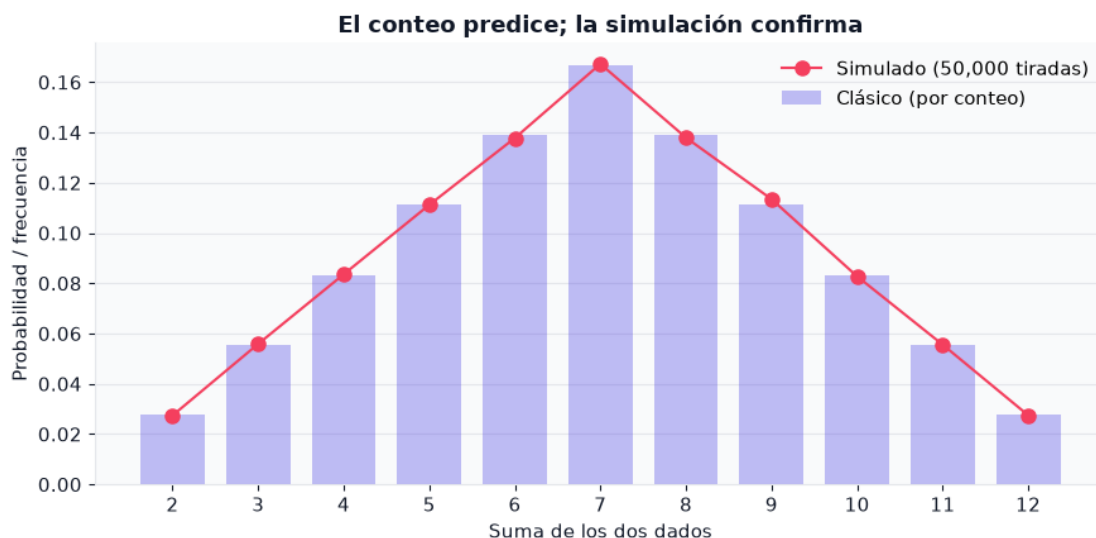
In [6]:

```
N = 50_000 # número de tiradas simuladas

d1 = rng.integers(1, 7, size=N) # N dados: enteros de 1 a 6
d2 = rng.integers(1, 7, size=N)
sumas_simuladas = d1 + d2 # suma elemento a elemento

conteo_sim = Counter(sumas_simuladas)
frec_obs = [conteo_sim[s] / N for s in sumas]

fig, ax = plt.subplots(figsize=(9, 4.5))
ax.bar(sumas, probs, color=INDIGO, alpha=0.35, label="Clásico (por conteo)")
ax.plot(sumas, frec_obs, "o-", color=CORAL, label=f"Simulado ({N:,} tiradas)")
ax.legend(frameon=False); plt.show()
```



Out [6] · Los puntos (simulación) caen casi exactamente sobre las barras (conteo).

CÓMO FUNCIONA EL CÓDIGO

- `rng.integers(1, 7, size=N)` genera N enteros aleatorios entre 1 y 6 **de una sola vez** (el 7 queda excluido). Trabajar con arreglos completos en lugar de un bucle —**vectorización**— es lo que hace a NumPy rápido.
- `d1 + d2` suma los dos arreglos **elemento a elemento**, produciendo las 50 000 sumas de golpe. Counter cuenta cuántas veces salió cada suma, y la **comprensión** la convierte en frecuencia relativa dividiendo entre N .
- En el gráfico, `ax.bar(..., alpha=0.35)` dibuja las barras teóricas semitransparentes y `ax.plot(..., "o-")` superpone los puntos simulados unidos por línea. La coincidencia visual es la verificación.

CONEXIÓN CON MACHINE LEARNING

Enumerar el espacio muestral y contar es el gesto fundamental del razonamiento probabilístico; todo lo demás lo generaliza.

El supuesto de equiprobabilidad es un **prior uniforme** —la misma idea que reaparece en la estimación bayesiana y MAP.

Que una distribución *con forma* (la triangular) emerja de piezas *uniformes* (dados planos) prefigura cómo los modelos generativos construyen distribuciones complejas a partir de componentes simples.

La pared contra la que choca el enfoque clásico

El enfoque clásico funcionó de maravilla con los dados. Pero mira **qué necesitó**: que supiéramos, de antemano y con certeza, que los 36 pares eran igual de posibles. Esa **simetría** fue la que nos dejó contar y dividir. Ahora una pregunta incómoda:

Lanzo una chincheta (tachuela) al aire. Puede caer con la punta hacia arriba ↑ o apoyada de lado ↱. ¿Cuál es la probabilidad de que caiga con la punta arriba?

¿1/2? **No hay ninguna razón para creerlo.** Una chincheta no es simétrica: su cabeza pesa distinto que su punta. No hay un espacio muestral parejo que enumerar, no hay simetría que explotar. El enfoque clásico se queda **mudo** —no porque lo apliquemos mal, sino porque no hay forma de aplicarlo. Y esto **es el caso normal en el mundo real**, no la excepción. Necesitamos otra manera de medir el azar: una que no cuente, sino que repita y observe. Esa es la Parte 2.

05 Parte 2 · El enfoque frecuentista: repetir y observar

Cambiamos por completo de gesto. En vez de contar posibilidades antes de tirar, el enfoque frecuentista **define** la probabilidad como la frecuencia relativa a la que ocurre un evento cuando repetimos el experimento muchísimas veces:

$$P(A) = \lim_{n \rightarrow \infty} \frac{\text{veces que ocurrió } A}{n}$$

En palabras llanas: **tira muchas veces, cuenta qué fracción salió, y esa fracción es tu probabilidad**. Con una trampa honesta: nunca llegamos a infinito, así que siempre obtenemos una **estimación**, nunca el valor perfecto. Esa limitación no es un defecto que esconder —es exactamente la condición en la que vive el machine learning. Lo probaremos en dos actos: primero en terreno conocido (una moneda, para ganar confianza), luego en terreno nuevo (la chincheta, donde es la única salida).

El gesto frecuentista, en una función

Definimos una función mínima que encarna la idea: repetir un experimento y devolver la **frecuencia relativa acumulada** del evento, tirada por tirada. Esa secuencia acumulada es lo que graficaremos.

In [7]:

```
def frecuencia_acumulada(resultados):
    '''Dada una secuencia de 0s y 1s (1 = ocurrió el evento),
    devuelve la fracción acumulada de 1s tras cada tirada.
    Ej.: [1,0,1,1] -> [1.0, 0.5, 0.667, 0.75]
    '''
    resultados = np.asarray(resultados)
    aciertos_acumulados = np.cumsum(resultados) # 1,1,2,3...
    tiradas = np.arange(1, len(resultados) + 1) # 1,2,3,4...
    return aciertos_acumulados / tiradas

print(frecuencia_acumulada([1, 0, 1, 1]))
```

Out:

```
[1.         0.5        0.66666667 0.75        ]
```

CÓMO FUNCIONA EL CÓDIGO

- `np.asarray(resultados)` convierte la entrada (una lista, por ejemplo) en un arreglo de NumPy para poder operar de forma vectorizada.
- `np.cumsum(resultados)` calcula la **suma acumulada**: en cada posición guarda el total de 1s vistos hasta ahí. Con `[1,0,1,1]` da `[1,1,2,3]`, es decir, los aciertos acumulados.
- `np.arange(1, len(...)+1)` genera el vector `1,2,3,...,N`: el número de tiradas hechas hasta cada punto.
- Dividir ambos arreglos **elemento a elemento** da la fracción de aciertos tras cada tirada. En el fondo, es la **media acumulada** de un vector de ceros y unos —y la media de una variable 0/1 es justo su frecuencia relativa.

Acto 1 — La moneda: ganar confianza

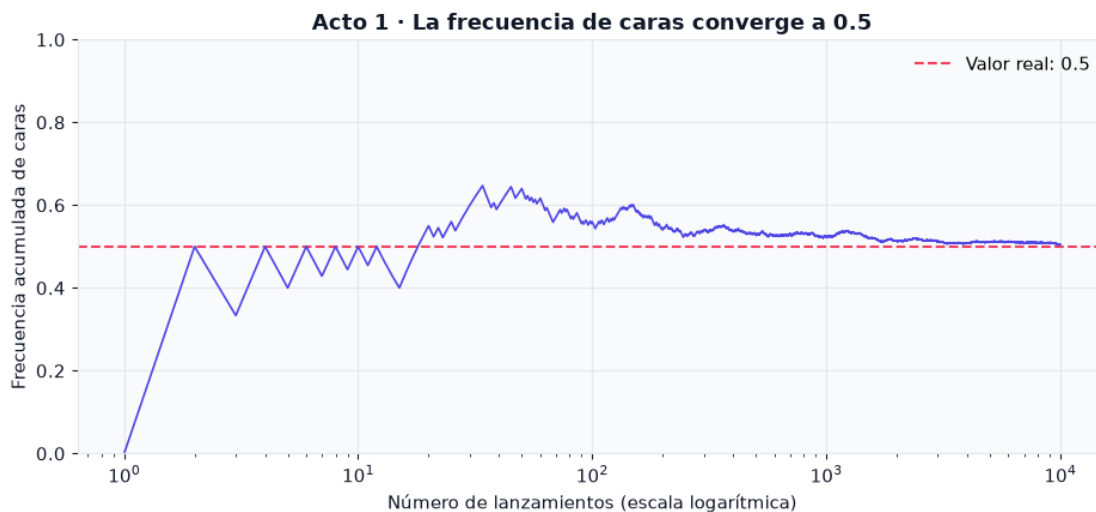
Empezamos con una moneda justa, donde *ya sabemos* la respuesta gracias al enfoque clásico: $P(\text{cara}) = 0.5$. Si el método frecuentista es bueno, debería converger solo a ese 0.5. Simulamos 10 000 lanzamientos y observamos cómo la frecuencia acumulada se estabiliza: al principio salta como loca; luego se calma.

In [8]:

```
N = 10_000
lanzamientos = rng.integers(0, 2, size=N) # 1 = cara, 0 = cruz
frec = frecuencia_acumulada(lanzamientos)

fig, ax = plt.subplots(figsize=(9.5, 4.5))
ax.plot(np.arange(1, N + 1), frec, color=INDIGO)
ax.axhline(0.5, color=CORAL, linestyle="--", label="Valor real: 0.5")
ax.set_xscale("log") # escala log en X: ver el caos inicial y la calma final
ax.set_ylim(0, 1); ax.legend(frameon=False); plt.show()

print(f"Estimación final: {frec[-1]:.4f} (el valor real es 0.5).")
```



Out [8] · La frecuencia acumulada de caras converge a 0.5 a medida que aumentan los lanzamientos.

CÓMO FUNCIONA EL CÓDIGO

- `rng.integers(0, 2, size=N)` genera N valores en $\{0, 1\}$ (el 2 queda excluido): 1 = cara, 0 = cruz, con probabilidad 0.5 cada uno.
- `ax.axhline(0.5, ...)` traza una línea horizontal de referencia en el valor verdadero, para comparar visualmente.
- `ax.set_xscale("log")` pone el eje X en **escala logarítmica**. Es un detalle importante: sin él, el caos de las primeras decenas de tiradas quedaría comprimido contra el borde izquierdo. La escala log "estira" los primeros lanzamientos y deja ver a la vez la turbulencia inicial y la calma final.
- `frec[-1]` toma el último elemento del arreglo (la estimación tras las 10 000 tiradas).

LO IMPORTANTE DEL ACTO 1

No es descubrir un número nuevo —ya sabíamos que era 0.5—, sino **comprobar que el procedimiento funciona**: repetir y observar devuelve la misma respuesta que el conteo clásico. Ahora podemos confiar en él para usarlo donde el clásico no llega.

Acto 2 — La chincheta: la necesidad

Ahora el caso que dejó mudo al enfoque clásico. Con la chincheta **no hay fórmula posible**: la única forma de conocer $P(\text{punta arriba})$ es lanzarla muchas veces y contar. Para simularlo con honestidad montamos un pequeño juego: la **naturaleza** tiene una probabilidad verdadera y **secreta**; nuestro experimento solo ve resultados (1 = punta arriba, 0 = de lado), nunca el número secreto; estimamos por frecuencia y **recién al final** revelamos el valor real para ver qué tan cerca quedamos.

In [9]:

```
# --- La naturaleza (secreto): NO mirar este número al razonar ---
P_VERDADERA = 0.62 # probabilidad real de "punta arriba"

N = 10_000
chinchetas = (rng.random(N) < P_VERDADERA).astype(int) # 0s y 1s
frec = frecuencia_acumulada(chinchetas)
estimacion = frec[-1]

print(f"Tras {N:,} lanzamientos, estimamos P(punta arriba) ~ {estimacion:.4f}")
```

Out:

```
Tras 10,000 lanzamientos, estimamos P(punta arriba) ~ 0.6191
```

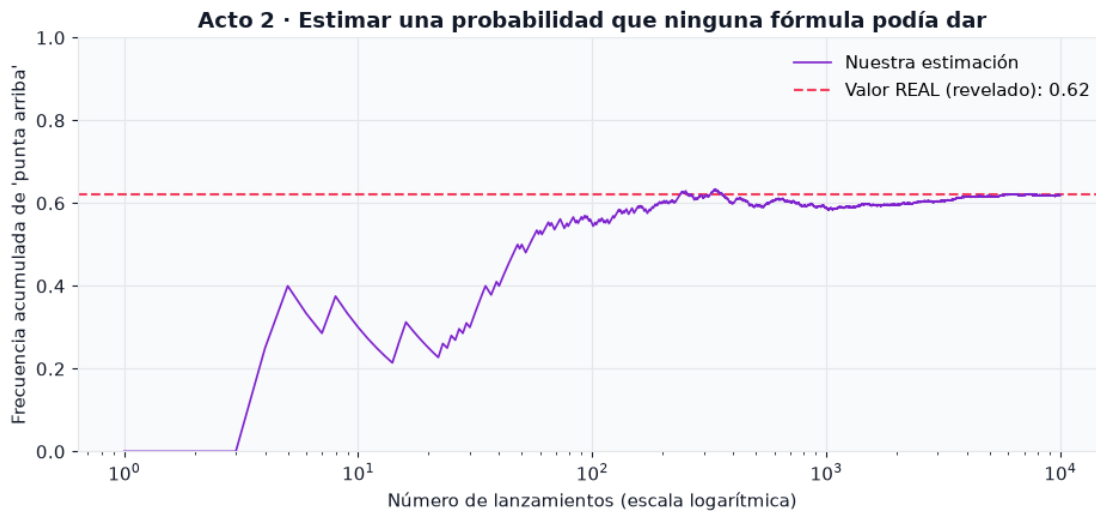
CÓMO FUNCIONA EL CÓDIGO – EL TRUCO DE BERNOULLI

- `rng.random(N)` genera N números reales uniformes en el intervalo $[0, 1)$. `rng.random(N) < P_VERDADERA` los compara con 0.62 y produce un arreglo de **booleanos**: cada valor es `True` (punta arriba) con probabilidad 0.62, porque exactamente el 62 % de los números uniformes cae por debajo de 0.62.
- `.astype(int)` convierte esos `True/False` en 1/0. Este es el patrón estándar para **simular un ensayo de Bernoulli** (un evento binario con probabilidad dada) de forma vectorizada.
- El código *nunca* usa `P_VERDADERA` para estimar: solo la emplea la “naturaleza” para generar los datos. Nuestra estimación sale únicamente de los 0s y 1s observados, igual que en un experimento real.

In [10]:

```
fig, ax = plt.subplots(figsize=(9.5, 4.5))
ax.plot(np.arange(1, N + 1), frec, color=PURPURA, label="Nuestra estimación")
ax.axhline(P_VERDADERA, color=CORAL, linestyle="--",
           label=f"Valor REAL (revelado): {P_VERDADERA}")
ax.set_xscale("log"); ax.set_ylim(0, 1); ax.legend(frameon=False); plt.show()

print(f"Estimación: {estimacion:.4f} | Real: {P_VERDADERA} | Error: {abs(estimacion - P_VERDADERA):.4f}")
```



Out [10] · La estimación (púrpura) converge al valor real 0.62 (coral), que ninguna fórmula podía dar.

Ese 0.62 **no salió de ninguna fórmula** —salió de los datos. Y así funciona exactamente el aprendizaje automático: la naturaleza tiene una distribución verdadera que nunca vemos directamente; solo tenemos muestras, y estimamos a partir de ellas.

El experimento de la fragilidad — ¿cuántos datos hacen falta?

La estimación frecuentista tiene una debilidad crítica que conviene sentir ahora: **con pocas repeticiones, es poco confiable**. Comparemos la estimación de la chincheta con distintos tamaños de muestra.

In [11]:

```
tamaños = [10, 100, 1_000, 10_000, 100_000]

estimaciones = []
for n in tamaños:
    muestra = (rng.random(n) < P_VERDADERA).astype(int)
    est = muestra.mean() # media = frecuencia de 1s
    estimaciones.append(est)
    print(f"{n:>8,} estimación={est:.4f} error={abs(est - P_VERDADERA):.4f}")
```

Out:

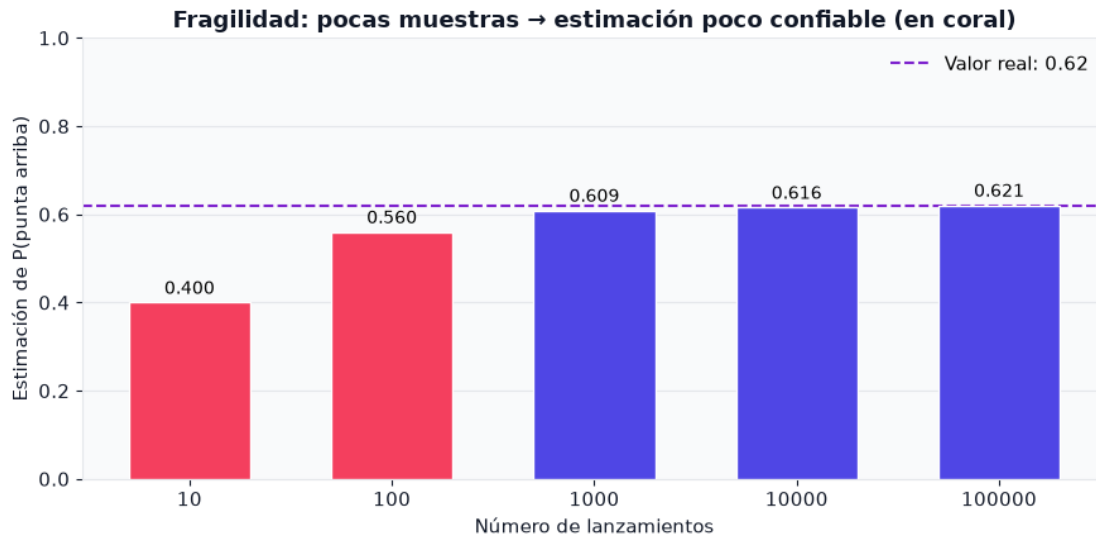
```
10 estimación=0.7000 error=0.0800
100 estimación=0.5900 error=0.0300
1,000 estimación=0.6260 error=0.0060
10,000 estimación=0.6203 error=0.0003
100,000 estimación=0.6197 error=0.0003
```

CÓMO FUNCIONA EL CÓDIGO

- El for recorre cinco tamaños de muestra. Para cada uno vuelve a aplicar el truco de Bernoulli y calcula la estimación con `muestra.mean()`, que es la media del arreglo de 0s y 1s —idéntica a su frecuencia relativa de unos.
- `estimaciones.append(est)` guarda cada resultado en una lista para graficarlo después. El **error** es el valor absoluto de la diferencia con `P_VERDADERA`.
- Observa el patrón: el error *no* baja de forma lineal, sino aproximadamente proporcional a $1/\sqrt{N}$. Multiplicar los datos por 100 reduce el error por un factor cercano a 10.

In [12]:

```
fig, ax = plt.subplots(figsize=(9, 4.5))
# coral si el error es grande (>0.03), indigo si es aceptable
colores = [CORAL if abs(e - P_VERDADERA) > 0.03 else INDIGO for e in estimaciones]
ax.bar([str(n) for n in tamaños], estimaciones, color=colores)
ax.axhline(P_VERDADERA, color=PURPURA, linestyle="--", label=f"Valor real: {P_VERDADERA}")
ax.set_ylim(0, 1); ax.legend(frameon=False); plt.show()
```



Out [12] · Pocas muestras → estimación poco confiable (en coral); muchas → sólida (en indigo).

CÓMO FUNCIONA EL CÓDIGO

- La **comprensión de lista con condicional** CORAL if ... else INDIGO decide el color de cada barra según si su error supera el umbral de 0.03. Es una forma compacta de “pintar de rojo lo poco confiable”.
- [str(n) for n in tamaños] convierte los tamaños a texto para usarlos como etiquetas categóricas del eje X (barras equiespaciadas en lugar de una escala numérica que aplastaría los valores pequeños).

Con $N = 10$ la estimación puede estar clamorosamente lejos; con $N = 100\,000$ es sólida como una roca. Ese es el precio del enfoque frecuentista: **necesita datos, y muchos**.

Un detalle sutil: la estimación misma es azarosa

Una última cosa que enseña muchísimo. Si repetimos *toda* la simulación varias veces con distintos “azares”, cada corrida traza un camino diferente hacia el valor real. Todas convergen al mismo sitio, pero por rutas ruidosas distintas.

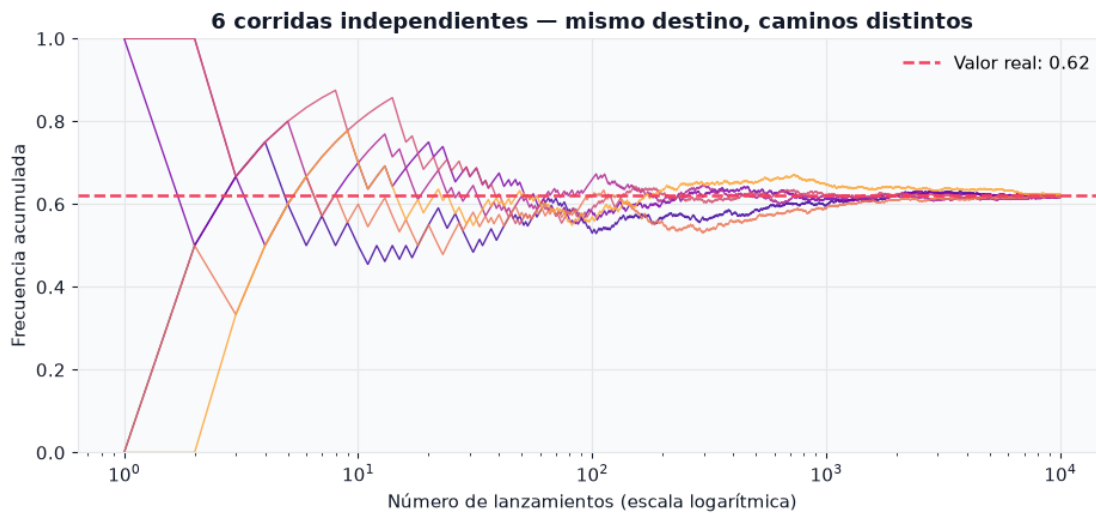
In [13]:

```
N = 10_000
n_corridas = 6

fig, ax = plt.subplots(figsize=(9.5, 4.5))
paleta = plt.cm.plasma(np.linspace(0.1, 0.8, n_corridas))

for k in range(n_corridas):
    rng_k = np.random.default_rng(k) # cada corrida, su propia semilla
    muestra = (rng_k.random(N) < P_VERDADERA).astype(int)
    ax.plot(np.arange(1, N + 1), frecuencia_acumulada(muestra),
            color=paleta[k], alpha=0.85)
```

```
ax.axhline(P_VERDADERA, color=CORAL, linestyle="--", label=f"Valor real: {P_VERDADERA}")
ax.set_xscale("log"); ax.set_ylim(0, 1); ax.legend(frameon=False); plt.show()
```



Out [13] · Seis corridas independientes: mismo destino (0.62), caminos distintos. Estimar es, en sí mismo, incierto.

CÓMO FUNCIONA EL CÓDIGO

- `np.random.default_rng(k)` crea un generador **nuevo** en cada vuelta del bucle, con semilla `k` distinta (0,1,...5). Así cada corrida produce una secuencia de azar independiente —seis “mundos paralelos” del mismo experimento.
- `plt.cm.plasma(np.linspace(0.1, 0.8, n_corridas))` toma seis colores equiespaciados del mapa plasma para distinguir las curvas.
- Todas las curvas terminan pegadas al 0.62, pero sus trayectorias iniciales difieren: el **límite es estable; el camino hacia él es azaroso**. Estimar desde una muestra finita conlleva, inevitablemente, **varianza**.

CONEXIÓN CON MACHINE LEARNING

Entrenar un modelo es esto. Estimar probabilidades y parámetros a partir de frecuencias observadas en los datos es, en esencia, lo que hace el aprendizaje. La **estimación por máxima verosimilitud** que veremos más adelante es, literalmente, “la frecuencia en los datos es tu mejor apuesta”, formalizada.

La chincheta es el mundo real. Nunca conoces las probabilidades verdaderas de la naturaleza; solo tienes una muestra y estimas. Vives en el Acto 2 permanentemente.

“Más datos ayuda” deja de ser un eslogan. La curva de convergencia es la justificación visual de por qué los datasets grandes importan.

La fragilidad prefigura el sobreajuste. Que la estimación con $N = 10$ sea pésima y ruidosa es la primera cara de un problema que perseguirá todo el curso: estimar desde pocos datos es peligroso y de **alta varianza**.

06 Síntesis: dos gestos para medir el azar

	Clásico (Parte 1)	Frecuentista (Parte 2)
Cómo obtiene P	Cuenta favorables / totales	Repite y observa la frecuencia
Cuándo	A priori, antes de tirar	A posteriori, tirando
Qué necesita	Simetría (equiprobabilidad)	Muchas repeticiones (datos)
Qué entrega	El valor exacto	Una estimación (nunca perfecta)
Se rompe cuando	No hay simetría (la chincheta)	Hay pocos datos (la fragilidad)
Ejemplo	Suma de dos dados	Moneda y chincheta

Los dos enfoques no compiten: se **complementan**. El clásico es el ideal exacto cuando la simetría existe; el frecuentista es la herramienta práctica cuando no. Y como en machine learning casi nunca hay simetría conocida, **vivimos casi siempre en el mundo frecuentista**: estimando desde datos, con toda la incertidumbre que eso implica.

07 Criterios de evaluación

Criterio	Descripción	Peso
Ejecución	El cuaderno corre completo sin errores y reproduce las figuras.	80 %
Interpretación	Explica cada resultado y lo conecta con las ideas de ML.	20 %

08 Referencias

- Laplace, P.-S. (1814). *Essai philosophique sur les probabilités*. (Origen de la definición clásica.)
- von Mises, R. (1928). *Probability, Statistics and Truth*. (Fundamento de la definición frecuentista.)
- Wasserman, L. (2004). *All of Statistics*. Springer. (Ley de los grandes números; máxima verosimilitud.)
- Downey, A. (2014). *Think Stats* (2.^a ed.). O'Reilly. (Estadística por simulación en Python.)
- NumPy Developers. *NumPy Reference — Random Generator*. numpy.org/doc.